Cassava Leaf Disease Classification

Ali Baki Türköz
Department of Computer Science
M.Sc. Program
Özyeğin University
Istanbul, Türkiye
baki.turkoz@ozu.edu.tr

Emre Öztürk
Department of Computer Science
M.Sc. Program
Özyeğin University
Istanbul, Türkiye
emre.ozturk.46760@ozu.edu.tr

***Abstract*—Plant disease detection affects crop yield and food security, and cassava diagnosis is difficult because several leaf diseases exhibit visually similar symptoms. This paper compares three families of image-classification approaches on the Cassava Leaf Disease Classification (CLDC) benchmark: classical raw-pixel baselines (SGD classifier, logistic regression, Extra Trees, and Random Forest), convolutional models (ResNet-50 and EfficientNet-B4), and a transformer model (ViT-B/16). Deep models are fine-tuned from ImageNet-pretrained weights under a reproducibility-oriented 3-fold stratified cross-validation protocol with early stopping, mixed-precision training, and class-weighting treated as a searchable option. An Optuna categorical search is followed by a fixed-configuration augmentation ablation. Across hyperparameter trials, ViT-B/16 attains the strongest macro-F1 (0.7803 ± 0.0105), outperforming the best ResNet-50 (0.7405 ± 0.0169) and EfficientNet-B4 (0.7274 ± 0.0123), while classical baselines remain below 0.21 macro-F1. With the best augmentation policy (`heavy_combo`), the final ViT-B/16 model reaches macro-F1 0.7921 ± 0.0068 and accuracy 0.8855 ± 0.0040, with the largest per-class gain ($+0.0282$ F1) on the rarest class.**

***Index Terms*—plant disease classification, cassava leaf disease, convolutional neural networks, vision transformer, transfer learning, Optuna, hyperparameter optimization, data augmentation, class imbalance, ablation study.**

I. Introduction

Agriculture remains a key pillar of global food security, and crop diseases continue to account for substantial yield losses worldwide. Among staple crops cultivated in tropical regions, cassava (*Manihot esculenta*) is particularly important: it provides a major source of carbohydrates for millions of producers in sub-Saharan Africa. However, cassava is highly susceptible to a number of viral, bacterial, and arthropod-mediated diseases, including Cassava Bacterial Blight (CBB), Cassava Brown Streak Disease (CBSD), Cassava Green Mite (CGM), and Cassava Mosaic Disease (CMD) [1]. Early and accurate identification of these diseases is essential for limiting losses, yet expert-driven manual inspection is expensive, slow, and difficult to scale.

Computer-vision-based automated disease detection offers a practical alternative. Deep convolutional neural networks (CNNs) have produced strong results in plant-disease classification under controlled laboratory conditions [2], [3], but generalization to images taken with mobile phones in farmers' fields—under varying illumination, complex backgrounds, motion blur, and label noise—remains an open challenge [4]. The Cassava Leaf Disease Classification (CLDC) dataset [1], released through a Kaggle competition, captures exactly these real-world difficulties. It contains roughly 21,397 images across five classes and exhibits severe class imbalance, with the CMD class accounting for more than 61% of the data while the rarest class (CBB) accounts for only about 5%.

Recent advances in computer vision have introduced two complementary directions worth investigating on this benchmark. On the convolutional side, EfficientNet [5] provides a principled compound-scaling rule that obtains higher accuracy than ResNet-style backbones [6] at comparable parameter budgets. On the architectural side, the Vision Transformer (ViT) [7] demonstrates that pure self-attention models, when pretrained on large-scale data, can match or exceed convolutional models on image classification. Whether these gains transfer to a noisy, imbalanced, in-the-wild agricultural domain such as cassava is a question that motivates this study.

In parallel, training-protocol decisions—such as the choice of optimizer, learning-rate schedule, loss function, and data-augmentation strategy—are known to have a comparable or even larger impact on final performance than the choice of backbone [8]–[10]. In settings with strong class imbalance and limited annotated samples, principled hyperparameter search and careful augmentation design are therefore not optional. This study addresses these considerations directly through an Optuna-based [11] categorical search over hyperparameters and an explicit augmentation ablation.

The main contributions of this work are the following:

- We provide a controlled, reproducibility-oriented comparison of three deep-learning architectures (ResNet-50, EfficientNet-B4, ViT-B/16) on CLDC under a unified 3-fold stratified cross-validation protocol, with class-weight handling, mixed-precision training, early stopping, and seed determinism.

- We include classical machine-learning baselines (SGD classifier, logistic regression, Extra Trees, and Random Forest) operating on grayscale flattened pixels with Principal-Component-Analysis (PCA) dimensionality reduction, to quantify the gap between raw-pixel classifiers and deep models on this task.
- We perform a categorical hyperparameter search using the Optuna framework with a Tree-structured Parzen Estimator (TPE) sampler and a median pruner, jointly searching over learning rate, weight decay, batch size, optimizer, loss function (including focal and label-smoothed variants), and learning-rate scheduler.
- We perform a data-augmentation ablation covering baseline, single-component, composite, and combined policies, and we analyze class weighting as a categorical option inside the hyperparameter search rather than as a separate matched-pair experiment.

The remainder of this paper is organized as follows. Section II reviews related work in plant-disease classification, transfer learning, and hyperparameter optimization. Section III describes the methodology in detail, including the dataset, models, training protocol, hyperparameter search procedure, and ablation design. Section IV presents the experimental results, the augmentation ablation, and per-class analyses. Section V concludes with a discussion of findings, limitations, and directions for future work.

II. Related Work

A. Plant-Disease Classification in the Wild

Early image-based plant-disease studies demonstrated that deep CNNs can reach very high accuracy under controlled imaging conditions. Mohanty *et al.* [2] trained AlexNet- and GoogLeNet-style models on PlantVillage and reported test accuracies above 99%, while Ferentinos [3] confirmed the strength of pretrained CNNs across multiple crop species. These results are important but not directly representative of deployment: Barbedo [4] showed that background clutter, illumination changes, occlusion, and annotation noise can substantially degrade recognition performance. The Cassava Leaf Disease Classification (CLDC) benchmark [1], [12] is therefore a stricter test case: images are acquired by farmers using mobile devices, symptoms are visually similar across classes, and the class distribution is highly imbalanced.

B. Architectures and Transfer Learning

ResNet-50 [6] remains a standard transfer-learning baseline because residual connections make deep CNN optimization stable. EfficientNet [5] improves the accuracy–efficiency trade-off through compound scaling of depth, width, and input resolution. ViT [7] replaces convolutional locality with self-attention over image patches and can be competitive when initialized from large-scale pretraining such as ImageNet [13]. However, recent evidence suggests that ImageNet progress does not always transfer monotonically to real-world datasets. Fang *et al.* [14] evaluated ImageNet-pretrained models on practical datasets including CLDC and reported that, on Cassava Leaf Disease, EfficientNet-B4, MobileNet, and ViT obtained 89.5%, 89.4%, and 88.8% accuracy respectively, while a stronger Kaggle ensemble reached 91.3% on the private leaderboard. This motivates evaluating backbone choice jointly with task-specific training choices rather than assuming that a larger or newer architecture will dominate.

C. Kaggle Competition Solutions and K-Fold Evaluation

The Kaggle CLDC competition provides a useful external reference point because submissions are evaluated by accuracy on a hidden test set and the final private leaderboard is computed on approximately 69% of that test set [15]. These numbers should not be interpreted as a direct replacement for our 3-fold cross-validation results, because many competition pipelines use larger ensembles, external 2019 cassava data, test-time augmentation (TTA), pseudo-labeling, relabeling, and leaderboard-driven model selection. They are nevertheless informative because the strongest public solutions identify which training strategies worked in the same data domain.

The documented first-place solution associated with `golddiggaz/lizh0019` used an ensemble of EfficientNet-B4, ResNeXt50, ViT, and MobileNet/CropNet-style components and reported roughly 91.3% accuracy on both public and private leaderboards [16]. The rank-2 leaderboard entry by Devon Stanfield achieved 0.9043 private leaderboard accuracy in the available leaderboard snapshot, and the associated public inference notebook uses Google's TensorFlow Hub CropNet cassava classifier [17]. The third-place TomYanabe solution is particularly relevant to our study because it reports a 5-fold StratifiedKFold protocol, ViT-B/16 variants at multiple input resolutions, 2020+2019 data, and TTA; its final submission reports CV 0.9075, public leaderboard 0.9075, and private leaderboard 0.9028 [18]. Krut Patel's ninth-place solution used stratified k-fold validation, ResNeXt50-32x4d and EfficientNet-B4 with soft voting, and TTA, reaching 0.895 accuracy on both public and private leaderboards [19]. Kozodoi's top-1% solution combined CNNs and Vision Transformers in a stacking ensemble and reports 91.06% test accuracy while ranking 14th among roughly 3,900 teams [20]. The kqu7 silver-medal solution used 5-fold StratifiedKFold, minority-sample weighting, heavy augmentation, ResNeXt50, and several EfficientNet variants; its individual model CV accuracies were 0.893–0.898, and its final ensemble with 8x TTA reached 90.11% public and 90.01% private leaderboard accuracy [21]. Finally, the vadimtimakin solution highlights the effect of relabeling and external data, reporting that soft relabeling increased EfficientNet-B4 CV from 0.899 to 0.915 and that the pipeline used 5-fold SWSL ResNeXt-101 predictions, 2019

data, Mendeley leaf images, MixUp/CutMix/FMix, and lightweight TTA [22].

Compared with these competition-oriented pipelines, our best post-augmentation single ViT-B/16 model reaches 88.55% ± 0.40% accuracy and 0.7921 ± 0.0068 macro-F1 under 3-fold stratified cross-validation, without ensembling, TTA, pseudo-labeling, relabeling, or external data. The gap to the strongest Kaggle systems is therefore expected: the leading entries optimize a full competition stack, while this paper isolates the effects of backbone family, hyperparameters, class-imbalance handling, and augmentation under one reproducible protocol.

D. Class Imbalance, Augmentation, and Optimization

Class imbalance is central to CLDC because CMD dominates the training set while the rarest class contributes only about 5% of the images. We therefore compare loss reweighting, focal loss [10], and label-smoothed cross-entropy [23]. Data augmentation is equally important for in-the-wild leaf images: geometric and photometric transforms model viewpoint and lighting variation, while Random Erasing [24] simulates occlusion and irrelevant background regions. Hyperparameter interactions across optimizer, scheduler, loss, and regularization make manual selection unreliable, so we use Optuna [11] with TPE search and median pruning. AdamW [8], SGDR/cosine scheduling [9], and mixed-precision training [25] are included to make the search computationally tractable. Classical linear and tree-based baselines, including Random Forests [26], are retained as lower-bound references for raw-pixel feature pipelines.

III. Methodology

A. Dataset

We use the Cassava Leaf Disease Classification (CLDC) dataset [1] released through a Kaggle competition. The dataset comprises approximately 21,397 field-acquired RGB images of cassava leaves, captured by farmers using mobile phones under uncontrolled illumination, varying backgrounds, partial occlusion, and varying image quality. Each image is labeled with one of five mutually exclusive classes. Fig. 1 reports the per-class image counts and proportions in the training metadata used in this work, and Fig. 2 shows one representative image from each class.

The dataset is highly imbalanced: the CMD class alone accounts for 61.49% of the training data, while the rarest class (CBB) accounts for only 5.08%. The ratio between the largest and smallest classes is approximately 12:1. This imbalance, together with the in-the-wild acquisition conditions, motivates the class-weighting, focal-loss, and augmentation strategies described below.

B. Evaluation Protocol

We use *3-fold stratified cross-validation* as the sole evaluation protocol; no separate held-out test split is used. Stratification preserves the per-class proportions of

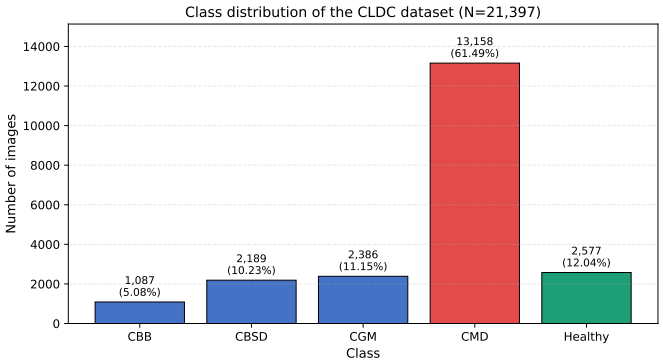


Fig. 1. Class distribution of the CLDC dataset used in this work. The Cassava Mosaic Disease (CMD) class accounts for 61.49% of the data, while the rarest class (CBB) accounts for only 5.08%; the ratio between the largest and smallest classes is approximately 12:1.



Fig. 2. Representative training images, one per class. The visual similarity between several disease classes, together with cluttered backgrounds and varying illumination, illustrates the difficulty of the CLDC benchmark.

Fig. 1 within each fold. For every experimental condition (every Optuna trial and every augmentation policy in the ablation), the full pipeline is repeated three times: training on two folds and evaluating on the held-out third. The reported metrics are the mean and standard deviation across the three folds. This corresponds to using out-of-fold validation predictions as a proxy for held-out test performance and provides a controlled alternative to a single fixed split.

For each fold, we record accuracy, macro-averaged precision, recall, and F1-score, the corresponding weighted-averaged metrics, and per-class precision, recall, and F1. We also record the full 5×5 confusion matrix and the training/validation loss and accuracy curves. Macro-averaged F1 (`f1_macro`) is the primary metric used both as the Optuna objective and as the headline comparison criterion, because it weighs all classes equally and is therefore appropriate for the strongly imbalanced CLDC class distribution. Accuracy, in contrast, is dominated by the majority CMD class and is reported as a secondary metric only.

C. Models

a) Classical baselines: For the classical pipeline, each image is converted to grayscale, resized to 64×64 pixels, scaled to $[0, 1]$, and flattened to a 4,096-dimensional feature vector. A single `StandardScaler` → `PCA(0.95 variance)` → classifier pipeline is then fit per fold. Four classifiers are evaluated: `sgd_classifier`, `logistic_regression`, `extra_trees`,

and `random_forest` [26]. These baselines deliberately use a small input resolution and a single global feature representation; they are intended as a lower-bound reference, not as competitive solutions.

b) Deep models: Three pretrained backbones are fine-tuned end-to-end. ResNet-50 [6] is loaded from `torchvision` with ImageNet-1k V2 weights; its final fully connected layer is replaced by a five-way classifier and the input resolution is set to 224×224. EfficientNet-B4 [5] is loaded with ImageNet-1k weights; the final linear layer of its classifier head is replaced and the input resolution is also set to 224×224 to keep memory cost comparable. ViT-B/16 [7] is loaded with ImageNet-1k weights and its classification head is replaced; following the architecture's design, the input resolution is set to 384×384. In all three cases, every parameter is set as trainable (no layer freezing); this corresponds to full fine-tuning.

D. Training Protocol

All deep models are trained with the same shared infrastructure, which differs only in the categorical hyperparameters chosen by Optuna. The base elements are the following.

a) Reproducibility: A single global seed is set for Python's `random` module, NumPy, PyTorch (CPU and CUDA), and `PYTHONHASHSEED`. Deterministic cuDNN kernels are enabled, the cuBLAS workspace is configured for determinism, and PyTorch's `use_deterministic_algorithms` flag is activated. DataLoader workers are seeded individually. These settings produce deterministic-settings-based runs at the cost of modest throughput.

b) Mixed precision: Training and inference are run under PyTorch's `torch.amp.autocast` with `float16` computation and a master copy of weights in `float32`, as introduced by Micikevicius *et al.* [25]. A `GradScaler` is used to prevent gradient underflow.

c) Class-imbalance handling: For each training fold, per-class weights $w_c = N/(C \cdot n_c)$ are computed from the fold's label distribution, where N is the number of training examples, $C = 5$ is the number of classes, and n_c is the count of class c. These weights are used either directly inside the loss (cross-entropy, label-smoothed cross-entropy, focal loss) or—in the unbalanced control condition—disabled. The `use_class_weights` flag is part of the ablation study described in Section III-F.

d) Loss functions: We compare three loss families:

- *Cross-entropy*, optionally with class weights depending on the `use_class_weights` setting.
- *Label-smoothed cross-entropy* [23] with smoothing parameter $\varepsilon \in \{0.05, 0.10, 0.20\}$.
- *Focal loss* [10] of the form $\mathrm{FL}(p_t) = -\alpha_t(1-p_t)^\gamma \log p_t$, with $\gamma \in \{1.0, 1.5, 2.0, 2.5\}$ and α_t taken from the class-balanced weights above.

TABLE I
Optuna categorical search space for deep models.

Hyperparameter	**Candidate values**
Model	`resnet50`, `efficientnet_b4`, `vit_b_16`
Learning rate	$\{1\times10^{-5}, 5\times10^{-5}, 1\times10^{-4}, 5\times10^{-4}, 1\times10^{-3}\}$
Weight decay	$\{1\times10^{-5}, 1\times10^{-4}, 1\times10^{-3}, 1\times10^{-2}\}$
Batch size	{16, 32, 64}
Loss	cross-entropy, label-smoothed CE, focal
Label smoothing	{0.05, 0.10, 0.20} (only for label-smoothed CE)
Focal γ	{1.0, 1.5, 2.0, 2.5} (only for focal loss)
Optimizer	`adamw`, `adam`, `sgd`
SGD momentum	{0.9, 0.95, 0.99} (only for SGD)
Scheduler	`plateau`, `cosine`, `cosine_warm`
Class weighting	{enabled, disabled}

e) Optimizers and schedulers: The search space includes AdamW [8], Adam, and SGD with Nesterov momentum, with momentum $\in \{0.9, 0.95, 0.99\}$ where applicable. The learning-rate schedule is chosen from `ReduceLROnPlateau`, cosine annealing, and cosine annealing with warm restarts (SGDR) [9]. Plateau scheduling uses factor 0.5 and patience 2; cosine variants use $\eta_{\min} = 10^{-7}$ and $T_0 = 5$ epochs with $T_{\mathrm{mult}} = 2$ for warm restarts.

f) Training loop, early stopping, and gradient clipping: Each fold is trained for at most 200 epochs with early stopping on validation loss (patience 20). At the end of each epoch the best model state (lowest validation loss) is retained in memory; this state is restored before producing the fold's final predictions. Gradients are clipped to an L_2 norm of 1.0 to stabilize fine-tuning of the larger ViT model.

E. Hyperparameter Search with Optuna

We use the Optuna framework [11] with a Tree-structured Parzen Estimator (TPE) sampler and a median pruner. All hyperparameters are categorical so that the same search space can be reused across runs and inspected directly in the resulting trial logs. Table I lists the search space. The Optuna objective is the mean macro-F1 across the three cross-validation folds of a trial; trials whose first-fold validation metric falls below the running median at the same epoch are pruned by the median pruner, allowing the budget to focus on promising configurations.

For every Optuna trial, the full 3-fold stratified cross-validation protocol of Section III-B is executed and every fold-level and aggregate metric is logged to a single trial-indexed CSV file, together with the corresponding categorical configuration. The complete configuration of any reported result can therefore be reconstructed unambiguously from the trial log.

F. Ablation Studies

To isolate the effect of design choices that are especially relevant to CLDC, we conduct a fixed-configuration aug-

mentation ablation and analyze class weighting as part of the Optuna search.

a) Augmentation ablation: After the Optuna search has identified one of the best-performing hyperparameter configurations, this configuration is fixed and re-trained under different augmentation policies. Beyond the standard *no-augmentation* baseline, the policies are grouped into single-component ablations (each of which isolates one transformation), composite recipes that combine a fixed set of standard transformations, and combined policies of varying intensity. Table II summarizes the policies and what each one applies. All augmentation pipelines share the same final preprocessing step (resize, conversion to tensor, ImageNet normalization), so any performance differences are attributable to the augmentation policy alone.

b) Class weighting: The `use_class_weights` flag is included in the Optuna search space as a categorical variable. Therefore, class-weighted and unweighted configurations are searched together, and the top-trial tables report both groups through the CW column. This design identifies whether class weighting survives model selection under the same search process; it is not treated as a separate matched-pair ablation with all other hyperparameters fixed.

G. Logging Convention

For every trial, a single row is appended to a master CSV file containing the full configuration (model, learning rate, weight decay, batch size, loss and its kwargs, optimizer and its kwargs, scheduler and its kwargs, augmentation policy, class-weighting flag, seed) and all aggregate and per-class metrics (mean and standard deviation across folds). Confusion matrices and training/validation curves are saved per trial. The reported metrics are taken from these experiment logs. Any language-editing assistance was restricted to grammar and readability polishing and did not generate or alter experimental results.

IV. Experimental Results

This section reports the results of the Optuna-driven hyperparameter search and the subsequent augmentation ablation. Hyperparameter trials were run for ResNet-50, EfficientNet-B4, and ViT-B/16 across class-weighted and unweighted configurations, and the classical baselines were evaluated under the same 3-fold stratified protocol. In addition, 15 augmentation settings are evaluated on top of the selected best deep-learning configuration, including the no-augmentation reference.

A. Classical Baselines

Table III reports the results of the four classical machine-learning baselines (`sgd_classifier`, `logistic_regression`, `extra_trees`, `random_forest`) operating on grayscale 64×64 flattened pixels with StandardScaler and PCA (95% variance) preprocessing. Macro-F1 values stay below 0.21 across all four methods, confirming that classical classifiers on raw global pixel statistics cannot capture the fine-grained leaf-texture cues that distinguish the five CLDC classes. Two effects are visible. First, tree-based methods (`extra_trees`, `random_forest`) achieve an apparently high accuracy (≈ 0.6149) only because they collapse to predicting the majority class CMD on almost every example: their macro-F1 of 0.1523 is essentially the chance level for a single dominant prediction, and per-class precision and recall confirm this degenerate behavior. Second, linear models (`logistic_regression`, `sgd_classifier`) trade some accuracy for slightly better balance across classes, which raises macro-F1 marginally but is still far below any deep model. These results form a lower-bound reference for the deep-learning experiments that follow.

B. Convolutional Models

Table IV reports the top-10 ResNet-50 and EfficientNet-B4 trials, ranked by macro-F1. The best ResNet-50 trial reaches macro-F1 0.7405 ± 0.0169 with accuracy 0.8521 ± 0.0085, an improvement of more than 0.54 macro-F1 points over the best classical baseline. Nine of the top ten CNN trials use ResNet-50; the best EfficientNet-B4 trial appears at rank 9 with macro-F1 0.7274 ± 0.0123, indicating that on this dataset and at the budgets explored by Optuna, the larger compound-scaled architecture does not consistently outperform the simpler ResNet baseline. The top six CNN trials, and nine of the top ten overall, were trained without explicit class-balanced loss weighting (CW column "×"); the only class-weighted entry in the top ten is at rank 7, which is consistent with the well-known difficulty of jointly tuning a class-reweighted loss and a strong optimizer on a strongly imbalanced classification problem.

C. Vision Transformer

Table V reports the top-10 ViT-B/16 trials. ViT clearly outperforms both CNN backbones: every entry in the top-10 is above 0.7700 macro-F1, the best trial achieves macro-F1 0.7803 ± 0.0105 and accuracy 0.8792 ± 0.0069, and trial ranked 3 achieves the highest accuracy overall (0.8802 ± 0.0070). The gap to the best CNN trial is approximately +0.04 macro-F1 ($\approx$ 5% relative improvement), which is a substantial margin given the difficulty of the dataset. As with the CNN results, the top ViT trials are dominated by unweighted training (eight of ten), while the two class-weighted entries at ranks 5 and 8 achieve slightly higher recall (0.7930 and 0.7911) at the cost of lower precision, indicating that class weighting on this task trades minority-class recall for majority-class precision rather than improving the macro objective monotonically.

D. Selected Best Model

The single best deep-learning configuration identified for downstream use is the rank-2 entry of Table V, a

TABLE II

THE 15 AUGMENTATION SETTINGS EVALUATED IN THE AUGMENTATION ABLATION. ALL PIPELINES END WITH IMAGENET NORMALIZATION.

Category	Policy	Description
Baseline	`none`	Resize and normalize only; no augmentation. Reference for all other policies.
Single component	`flip_only`	Horizontal/vertical flip and transpose only (spatial symmetry).
	`color_only`	Color jitter and random grayscale only (no geometry).
	`contrast_only`	Contrast jitter only.
	`rotate_only`	Rotation only (no flip, no crop).
	`blur_noise_light`	Light Gaussian blur (sensor-noise simulation).
	`perspective_light`	Light perspective distortion (camera-tilt simulation).
Composite recipes	`comp_resnext`	RandomResizedCrop, Transpose, H/V flip, ShiftScaleRotate.
	`comp_vit`	Above plus HSV jitter, brightness/contrast adjustments, CoarseDropout/Cutout.
	`comp_3rd`	RandomResizedCrop, Transpose, H/V flip, ShiftScaleRotate only (minimal composite).
Combined	`heavy_combo`	Full geometry + full color + Random Erasing.
	`cutout_double`	Double occlusion: Cutout + CoarseDropout.
	`regularization_light_combo`	Light crop + flip + small Random Erasing.
	`clahe_enhanced`	CLAHE histogram equalization with light color and geometry.
	`color_light_combo`	Flip + ColorJitter; no geometric distortion.

TABLE III

CLASSICAL MACHINE-LEARNING BASELINE RESULTS ON CLDC (3-FOLD STRATIFIED CV; RAW 64×64 GRAYSCALE FLATTEN → STANDARDSCALER → PCA (95% VARIANCE) → CLASSIFIER). ALL VALUES ARE MEAN±STD ACROSS THE THREE FOLDS.

Classifier	F1 (macro)	Accuracy	Prec. (macro)	Recall (macro)
`sgd_classifier`	0.2012 ± 0.0064	0.3088 ± 0.0145	0.2133	0.2216
`logistic_regression`	0.1898 ± 0.0037	0.2428 ± 0.0043	0.2167	0.2270
`extra_trees`	0.1523 ± 0.0000	0.6149 ± 0.0001	0.1230	0.2000
`random_forest`	0.1523 ± 0.0000	0.6149 ± 0.0000	0.1230	0.2000

ViT-B/16 trial with macro-F1 0.7800 ± 0.0087 and accuracy 0.8797 ± 0.0061. Although the rank-1 entry achieves marginally higher macro-F1 (0.7803), the rank-2 trial is preferred for two complementary reasons. First, its train–validation gap on macro-F1 ($0.8409 - 0.7800 = 0.0609$) is the smallest among the leading trials, indicating a model that generalizes more cleanly to held-out folds. Second, its `ReduceLROnPlateau` schedule with fixed seed 42 produced numerically stable behavior across all three folds, making it the most suitable starting point for the data-augmentation ablation. The complete configuration of this trial is summarized in Table VI, with the per-class metrics shown in Fig. 3 and the corresponding confusion matrix in Fig. 4 and training curves in Fig. 5.

A notable observation is that the best validation epoch falls in the range $[1, 2]$ on all three folds, despite the 200-epoch budget and patience 20. This indicates that fine-tuning a fully ImageNet-pretrained ViT on CLDC saturates extremely quickly: after roughly two epochs of full-network adaptation, additional epochs only increase the train–validation gap. The current best deep-learning trial reaches accuracy ≈ 0.88, and targeted augmentation strategies reported in Section IV-F further reduce the train–validation gap and improve the macro objective.

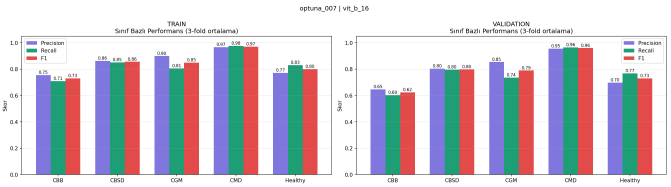


Fig. 3. Per-class precision, recall, and F1 of the selected best deep-learning trial (mean across 3 folds). The model handles CMD almost perfectly ($F_1 \approx 0.96$) but is weakest on CBB ($F_1 \approx 0.63$), the rarest class.

E. Per-Class Behavior and Hardest Class

Across every leading trial in Tables IV and V, the per-class metrics follow the same pattern. CMD is essentially solved ($F_1 \approx 0.96$), CBSD and CGM reach the high 0.75–0.80 range, Healthy lies near 0.72, and CBB is consistently the hardest class with $F_1 \approx 0.63$ at best. Two factors are responsible. First, CBB has only 1,087 training images (about one-twelfth of CMD), so even balanced reweighting cannot fully compensate for the limited intra-class diversity. Second, the visual symptoms of CBB (angular

TABLE IV

TOP-10 CNN TRIALS (RESNET-50 AND EFFICIENTNET-B4) ON CLDC, RANKED BY MACRO-F1 OVER A 3-FOLD STRATIFIED CV. "CW" INDICATES WHETHER CLASS-BALANCED LOSS WEIGHTING WAS USED. VALUES ARE MEAN±STD ACROSS FOLDS.

Rank	Model	CW	F1 (macro)	Accuracy	Prec. (macro)	Recall (macro)
1	`resnet50`	×	0.7405 ± 0.0169	0.8521 ± 0.0085	0.7576	0.7298
2	`resnet50`	×	0.7375 ± 0.0161	0.8508 ± 0.0081	0.7530	0.7290
3	`resnet50`	×	0.7345 ± 0.0064	0.8458 ± 0.0040	0.7400	0.7311
4	`resnet50`	×	0.7311 ± 0.0104	0.8459 ± 0.0046	0.7445	0.7210
5	`resnet50`	×	0.7310 ± 0.0080	0.8421 ± 0.0059	0.7353	0.7298
6	`resnet50`	×	0.7303 ± 0.0125	0.8454 ± 0.0059	0.7434	0.7213
7	`resnet50`	✓	0.7299 ± 0.0161	0.8365 ± 0.0171	0.7197	0.7502
8	`resnet50`	×	0.7281 ± 0.0128	0.8416 ± 0.0090	0.7370	0.7248
9	`efficientnet_b4`	×	0.7274 ± 0.0123	0.8494 ± 0.0059	0.7514	0.7112
10	`resnet50`	×	0.7271 ± 0.0094	0.8415 ± 0.0045	0.7357	0.7254

TABLE V

TOP-10 VISION TRANSFORMER (VIT-B/16) TRIALS ON CLDC, RANKED BY MACRO-F1 OVER A 3-FOLD STRATIFIED CV. "CW" INDICATES WHETHER CLASS-BALANCED LOSS WEIGHTING WAS USED. VALUES ARE MEAN±STD ACROSS FOLDS.

Rank	Model	CW	F1 (macro)	Accuracy	Prec. (macro)	Recall (macro)
1	`vit_b_16`	×	0.7803 ± 0.0105	0.8792 ± 0.0069	0.7880	0.7764
2	`vit_b_16`	×	0.7800 ± 0.0087	0.8797 ± 0.0061	0.7908	0.7733
3	`vit_b_16`	×	0.7758 ± 0.0122	0.8802 ± 0.0070	0.7936	0.7660
4	`vit_b_16`	×	0.7750 ± 0.0073	0.8755 ± 0.0035	0.7919	0.7705
5	`vit_b_16`	✓	0.7747 ± 0.0122	0.8684 ± 0.0080	0.7596	0.7930
6	`vit_b_16`	×	0.7739 ± 0.0061	0.8772 ± 0.0029	0.7864	0.7640
7	`vit_b_16`	×	0.7728 ± 0.0178	0.8744 ± 0.0081	0.7913	0.7654
8	`vit_b_16`	✓	0.7719 ± 0.0053	0.8654 ± 0.0024	0.7576	0.7911
9	`vit_b_16`	×	0.7712 ± 0.0066	0.8755 ± 0.0063	0.7818	0.7640
10	`vit_b_16`	×	0.7700 ± 0.0062	0.8747 ± 0.0033	0.7789	0.7620



Fig. 4. Confusion matrix of the selected best deep-learning trial (out-of-fold predictions aggregated across 3 folds). Most off-diagonal mass for CBB and Healthy is absorbed into CMD, the dominant class.

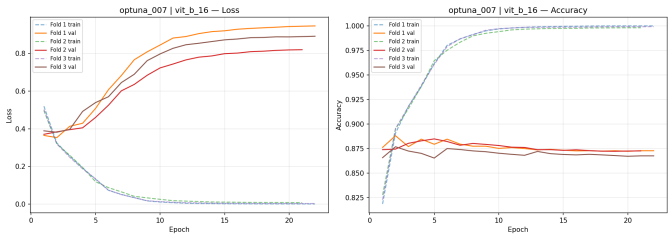


Fig. 5. Training and validation loss/accuracy curves for the selected best trial across the three folds. Convergence is extremely fast (best epoch within $[1, 2]$ on every fold), after which validation loss diverges from training loss; the early-stopping criterion restores the best validation checkpoint before final evaluation.

leaf spots) overlap with both Healthy and CGM under varying illumination, which puts a hard ceiling on the achievable per-class F1 without targeted augmentation. These observations directly motivate the augmentation ablation reported next.

F. Augmentation Ablation

To quantify the effect of data augmentation independently of the hyperparameter search, the selected best ViT-B/16 configuration of Table VI is held fixed and re-trained under 15 augmentation settings, including the no-augmentation baseline. All other settings (learning rate, weight decay, batch size, optimizer, scheduler, class-weighting flag, seed, input resolution, epoch budget) are identical across runs, so any observed differences in validation metrics are attributable to the augmentation policy alone. The no-augmentation row in this ablation is a retrained baseline produced with the fixed ablation script, yielding macro-F1 0.7812 ± 0.0096; this is close to the selected hyperparameter-search trial in Table VI (0.7800 ± 0.0087). Table VII reports the full ranking and Fig. 6 visualizes the results.

Several observations follow from these results. First,

TABLE VI
CONFIGURATION OF THE SELECTED BEST DEEP-LEARNING TRIAL (VIT-B/16, USED AS THE STARTING POINT FOR THE AUGMENTATION ABLATION).

Hyperparameter	**Value**
Backbone	`vit_b_16` (ImageNet-1k pretrained)
Input resolution	384×384
Learning rate	5×10^{-4}
Weight decay	1×10^{-3}
Batch size	16
Loss	cross-entropy (`kwargs: {}`)
Class weighting	disabled (`use_class_weights = False`)
Optimizer	SGD with Nesterov momentum 0.95
Scheduler	`ReduceLROnPlateau` (factor 0.5, patience 2, $\eta_{\min}$
Augmentation	`none`
Epoch budget	200 (early-stop patience 20)
Mixed precision	enabled (AMP)
Seed	42
Best epoch per fold	[2, 1, 2]
Val. F1 (macro)	0.7800 ± 0.0087
Val. Accuracy	0.8797 ± 0.0061
Val. Precision (macro)	0.7908
Val. Recall (macro)	0.7733
Train F1 (macro)	0.8409
Train–Val gap (F1)	0.0609

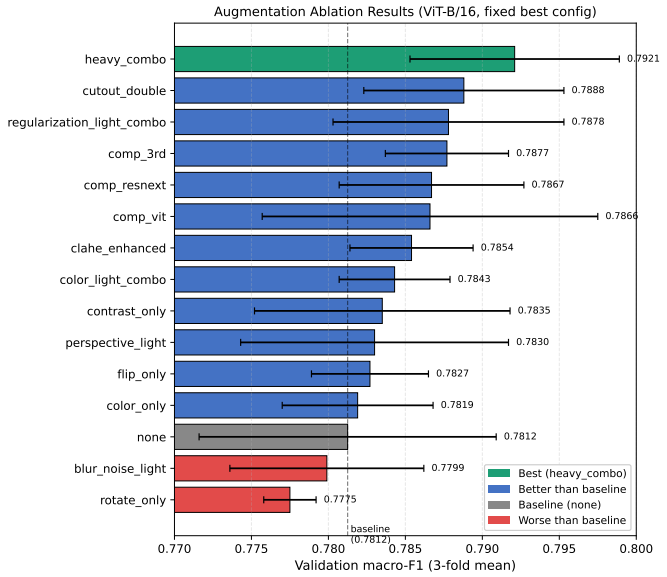


Fig. 6. Augmentation ablation: validation macro-F1 of the selected best ViT-B/16 configuration under each augmentation policy. Error bars denote one standard deviation across the 3 folds. The dashed vertical line marks the `none` baseline; bars are colored to indicate whether each augmentation improved (blue), degraded (red), or matched (gray for baseline, green for best) the baseline performance.

12 of the 14 non-baseline augmentation policies (every entry except `blur_noise_light` and `rotate_only`) outperform the no-augmentation baseline on macro-F1, which directly confirms the project hypothesis that data augmentation is beneficial on CLDC. Second, the largest gains come from combined policies that mix geometric, photometric, and occlusion-style transformations: `heavy_combo` (rank 1), `cutout_double` (rank 2), and `regularization_light_combo` (rank 3) all combine multiple augmentation families and form the top of the table. Third, single-component policies that include only one weak transformation (e.g. `rotate_only`, `blur_noise_light`) actually reduce macro-F1 relative to baseline, showing that weak, isolated augmentations can act as noise without compensating regularization benefits. Fourth, the train–validation gap of the best augmentation (0.0396 for `heavy_combo`) is meaningfully smaller than that of the no-augmentation baseline (0.0604), indicating that augmentation reduces overfitting rather than simply shifting the operating point.

1) Selected Best Augmentation: `heavy_combo`: The best augmentation policy is `heavy_combo`, with macro-F1 0.7921 ± 0.0068 and accuracy 0.8855 ± 0.0040. This corresponds to an absolute improvement of +0.0109 macro-F1 over the no-augmentation baseline (a +1.4% relative gain), achieved while simultaneously reducing the train–validation gap from 0.0604 to 0.0396 and pushing the best-epoch range from $[1, 2]$ in the baseline to $[7, 11]$ across folds, which indicates that the model is now able to make productive use of the additional training budget rather than overfitting after one or two epochs. The combination of full geometric transformations, full color jittering, and Random Erasing therefore acts both as a regularizer (reducing the train–validation gap) and as a data-diversity amplifier (extending useful training time).

Figure 7 explains the mechanism behind the augmentation gain by comparing directly with the pre-augmentation curves in Fig. 5. Without augmentation, the model memorizes the training folds rapidly: training loss approaches zero, training accuracy approaches 100%, validation loss rises monotonically after the first few epochs, and validation accuracy saturates around 87–88%. With `heavy_combo`, training remains deliberately harder: training loss decreases more slowly and remains around 0.25 at the end of training, while training accuracy stabilizes near 91% instead of reaching perfect memorization. More importantly, validation loss stays approximately stable around 0.35–0.38 rather than increasing toward 0.8–0.9, and validation accuracy remains in the 88–89% band. This behavior confirms that the augmentation policy improves generalization by suppressing memorization and by keeping the optimization trajectory useful for more epochs, rather than by merely increasing the apparent validation accuracy at a single early checkpoint.

The per-class behavior of `heavy_combo` (Fig. 8) is the most informative aspect of the augmentation ablation. All five classes gain F1, but the gains are highly asymmetric: CBB, the rarest class and the one with the lowest baseline F1, gains the most (+0.0282 F1), followed by CGM

TABLE VII

AUGMENTATION ABLATION RESULTS ON CLDC. ALL TRIALS USE THE SELECTED BEST VIT-B/16 CONFIGURATION FROM TABLE VI AS A FIXED STARTING POINT; ONLY THE AUGMENTATION POLICY VARIES. VALUES ARE MEAN±STD ACROSS 3 FOLDS. "ΔF1" IS THE CHANGE IN MACRO-F1 WITH RESPECT TO THE `none` BASELINE. ROWS ARE RANKED BY MACRO-F1.

Rank	Augmentation	F1 (macro)	Accuracy	Prec. (macro)	Recall (macro)	ΔF1 vs none
1	`heavy_combo`	0.7921 ± 0.0068	0.8855 ± 0.0040	**0.7980**	**0.7890**	+0.0109
2	`cutout_double`	0.7888 ± 0.0065	0.8835 ± 0.0031	0.7923	0.7864	+0.0075
3	`regularization_light_combo`	0.7878 ± 0.0075	0.8843 ± 0.0048	0.8005	0.7786	+0.0065
4	`comp_3rd`	0.7877 ± 0.0040	0.8835 ± 0.0028	0.7959	0.7821	+0.0064
5	`comp_resnext`	0.7867 ± 0.0060	0.8844 ± 0.0045	0.8033	0.7749	+0.0054
6	`comp_vit`	0.7866 ± 0.0109	0.8848 ± 0.0039	0.7995	0.7806	+0.0053
7	`clahe_enhanced`	0.7854 ± 0.0040	0.8850 ± 0.0016	0.8041	0.7718	+0.0041
8	`color_light_combo`	0.7843 ± 0.0036	0.8832 ± 0.0024	0.7976	0.7728	+0.0030
9	`contrast_only`	0.7835 ± 0.0083	0.8811 ± 0.0058	0.7929	0.7773	+0.0022
10	`perspective_light`	0.7830 ± 0.0087	0.8828 ± 0.0049	0.7975	0.7719	+0.0018
11	`flip_only`	0.7827 ± 0.0038	0.8818 ± 0.0034	0.7936	0.7746	+0.0014
12	`color_only`	0.7819 ± 0.0049	0.8813 ± 0.0030	0.7985	0.7688	+0.0007
13	`none` (baseline)	0.7812 ± 0.0096	0.8800 ± 0.0063	0.7927	0.7740	—
14	`blur_noise_light`	0.7799 ± 0.0063	0.8814 ± 0.0034	0.7984	0.7654	−0.0013
15	`rotate_only`	0.7775 ± 0.0017	0.8805 ± 0.0022	0.7934	0.7674	−0.0038

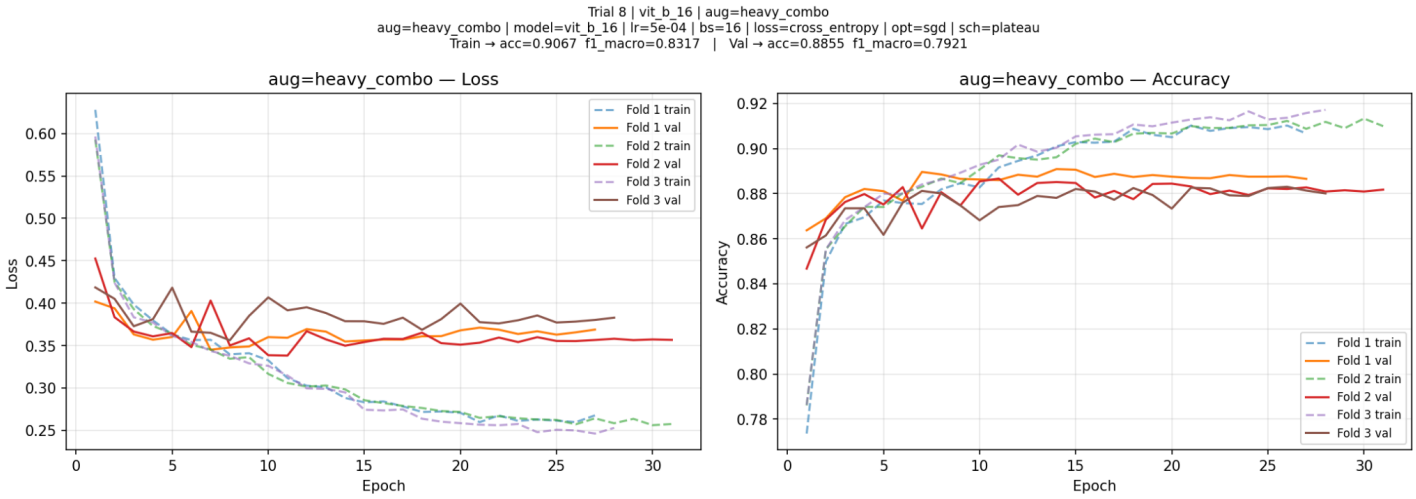


Fig. 7. Training and validation curves for the selected best augmentation policy, `heavy_combo`, using the same ViT-B/16 configuration as the no-augmentation baseline. In contrast to Fig. 5, training loss no longer collapses to zero and validation loss no longer diverges upward after the first few epochs. The curves instead show slower training convergence, validation-loss stabilization around the mid-0.3 range, and validation accuracy near 88–89% across folds.

(+0.0116) and CBSD (+0.0074). The two best-performing classes at baseline (CMD with $F_1 \approx 0.96$ and Healthy with $F_1 \approx 0.73$) gain the least (+0.0024 and +0.0046 respectively). This is the desired pattern for an imbalanced dataset: augmentation reallocates the model's representational capacity toward the harder, minority class without sacrificing performance on the majority class.

The confusion matrices in Fig. 9 make the per-class effect concrete. Under the no-augmentation baseline the model confuses CBB with CMD and CGM relatively often, and CBSD with CMD; `heavy_combo` reduces both off-diagonal blocks. The dominant diagonal cell for CMD remains essentially unchanged, confirming that the majority class is unaffected by the augmentation.

V. DISCUSSION AND CONCLUSION

This work presented a controlled, reproducibility-oriented comparison of three families of image classification approaches for the Cassava Leaf Disease Classification benchmark—classical machine-learning baselines, two convolutional architectures (ResNet-50, EfficientNet-B4), and the Vision Transformer (ViT-B/16)—together with an Optuna-driven categorical hyperparameter search and an augmentation ablation. We organize the discussion around five findings that emerge from the experiments and from the comparison with public Kaggle solutions.

a) Architecture matters, but less than expected at scale: Among the deep architectures, ViT-B/16 produces

TABLE VIII

COMPARISON WITH PUBLICLY DOCUMENTED KAGGLE CLDC SOLUTIONS. REPORTED LEADERBOARD/TEST RESULTS ARE NOT STRICTLY IDENTICAL TO OUR 3-FOLD CV BECAUSE THE KAGGLE SYSTEMS GENERALLY USE EXTERNAL DATA, TTA, ENSEMBLING, RELABELING, OR HIDDEN-TEST MODEL SELECTION. "PP" DENOTES PERCENTAGE POINTS RELATIVE TO OUR BEST POST-AUGMENTATION ACCURACY OF 88.55%.

Work	Main setup	Validation / selection	Reported best result	Difference vs. ours
lizh0019 / golddiggaz [16]	Ensemble of EfficientNet-B4, ResNeXt50, ViT, and MobileNet/CropNet-style models	OOF ensemble search; competition selection	≈ 91.3% public/private LB, rank 1	+2.75 pp vs. our 88.55% CV accuracy
Devon Stanfield [17]	TensorFlow Hub CropNet cassava classifier inference notebook	Public notebook; leaderboard selection	90.43% private LB, rank 2 in available snapshot	+1.88 pp; method details less complete
TomYanabe [18]	ViT-B/16 at 224/384 input resolutions, 2020+2019 data, TTA=5	5-fold StratifiedKFold	CV 90.75%, public 90.75%, private 90.28%, rank 3	+1.73 pp; closest high-rank k-fold comparison
Krut Patel [19]	ResNeXt50-32x4d + EfficientNet-B4 soft-voting ensemble with TTA	Stratified k-fold CV	89.5% public/private LB, rank 9	+0.95 pp; similar k-fold design but ensemble/TTA used
Kozodoi [20]	CNN + Vision Transformer stacking ensemble	Base-model training plus stacking	91.06% reported test accuracy, top 1% / rank 14	+2.51 pp; ensemble and stacking not used here
kqu7 [21]	ResNeXt50 + EfficientNet variants, class sampling, heavy augmentation, 8x TTA	5-fold StratifiedKFold	CV 89.3–89.8%; public 90.11%, private 90.01%	+1.46 pp on private LB; very close protocol but ensemble/TTA used
This work	Single ViT-B/16 with Optuna-selected hyperparameters and `heavy_combo` augmentation	3-fold StratifiedKFold	Accuracy 88.55% ± 0.40%, macro-F1 0.7921 ± 0.0068	Baseline for controlled single-model study

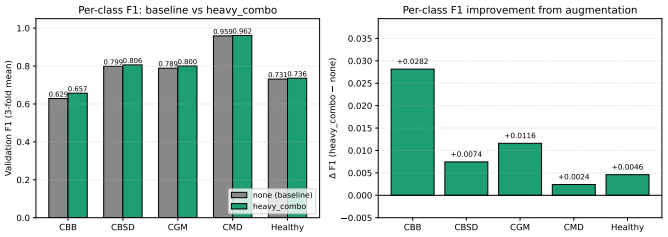
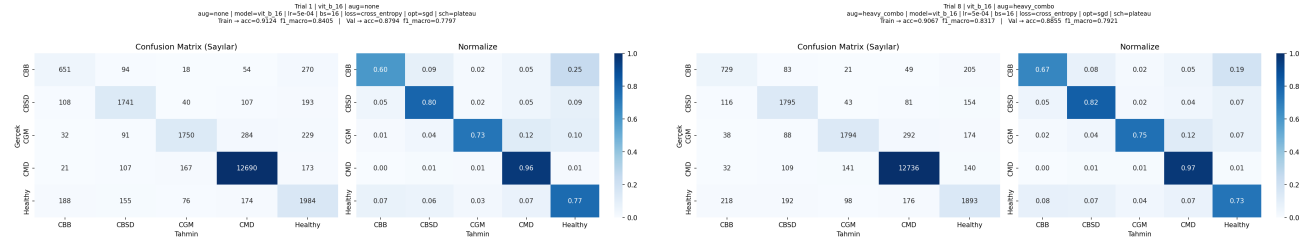


Fig. 8. Per-class effect of the best augmentation policy. Left: per-class validation F1 under the no-augmentation baseline (gray) and under `heavy_combo` (green). Right: per-class change ΔF_1 from baseline to `heavy_combo`; all five classes improve, and the largest gain (+0.0282) is on the rarest and previously hardest class, CBB.

the strongest results: every entry in the ViT top-10 (Table V) surpasses the best ResNet-50 trial in Table IV, and the absolute gap between the best ViT trial (0.7803 macro-F1) and the best CNN trial (0.7405 macro-F1) is approximately +0.04 macro-F1, or about 5% relative. This is a substantial margin given the difficulty of the dataset and is consistent with the broader literature showing that ImageNet-pretrained transformers transfer well to natural-image tasks. However, the gap between the two CNN backbones is much smaller and not monotonic: nine of the top-10 CNN trials use ResNet-50 and the best EfficientNet-B4 trial appears only at rank 9, suggesting that the larger compound-scaled architecture does not consistently outperform the simpler ResNet baseline at the hyperparameter budgets we explored. In practice this means that for a given compute budget, the choice between competitive CNN backbones is dominated by hyperparameter and augmentation choices rather than by architectural depth or scaling alone. Input resolution differs across architectures; part of the ViT advantage may be attributable to higher resolution.

b) Classical baselines are inadequate on raw pixels: The classical pipeline (grayscale 64×64 flatten → StandardScaler → PCA at 95% variance → classifier) yields macro-F1 scores below 0.21 across all four classifiers (Table III), more than 0.53 macro-F1 points below the best deep model. Tree-based methods reach an apparently high accuracy (≈ 0.6149) only because they collapse to predicting the majority CMD class on almost every example, as evidenced by their degenerate per-class precision and recall. This confirms that the discriminative cues for CLDC live in fine-grained local leaf texture rather than in global pixel statistics, and that any reasonable classifier on raw pixels alone cannot solve this task.

c) Hyperparameter choices and class weighting: The Optuna search reveals two consistent patterns. First, the top deep-learning trials come from the combined search over both class-weighted and unweighted configurations; the CW column in Tables IV and V makes this explicit. The highest-ranked models are dominated by unweighted training (eight of ten ViT trials, nine of ten CNN trials), while class-weighted candidates generally trade minority-class recall for majority-class precision rather than improving the macro objective monotonically. Second, on this dataset the best ViT configurations consistently used SGD with Nesterov momentum and a plateau or cosine warm-restart schedule rather than AdamW, despite AdamW being the de-facto default for transformer fine-tuning. We interpret this as a property of the dataset rather than a property of the architecture: with strongly imbalanced



(a) `none` (baseline) (b) `heavy_combo` (best)

Fig. 9. Out-of-fold confusion matrices for the no-augmentation baseline (a) and the best augmentation policy `heavy_combo` (b), both evaluated with the selected best ViT-B/16 configuration. The reduction of off-diagonal mass for CBB and CBSD (rows 0 and 1) is the dominant source of the macro-F1 improvement.

labels and noisy real-world images, the implicit regularization of SGD appears to outperform the more aggressive adaptation of AdamW. A practical consequence is that for similar agricultural in-the-wild benchmarks, hyperparameter assumptions imported from clean academic datasets should be questioned and re-tuned.

d) Augmentation gains concentrate on the minority class: The augmentation ablation reported in Section IV-F is, in our view, the most informative experiment in this work. Twelve of the fourteen non-baseline augmentation policies improve macro-F1 over the no-augmentation baseline (Table VII), the best policy (`heavy_combo`) achieves macro-F1 0.7921 ± 0.0068 versus the baseline's 0.7812 ± 0.0096, and the per-class breakdown in Fig. 8 shows that the largest absolute gain (+0.0282 F1) is on CBB, the rarest and previously hardest class, while the majority CMD class is essentially unaffected (+0.0024 F1). The training curves make the same conclusion visible at the optimization level: Fig. 5 shows near-immediate memorization without augmentation, whereas Fig. 7 shows that `heavy_combo` slows down training convergence, prevents the validation-loss explosion, and keeps validation accuracy stable for a longer useful training window. The train–validation gap drops from 0.0604 to 0.0396, and the best-epoch indices move from $[1, 2]$ on the baseline to $[7, 11]$ on `heavy_combo`, indicating that augmentation both reduces overfitting and allows the model to make productive use of additional training time. In contrast, single-component policies that include only one weak transformation can reduce macro-F1 below baseline (`rotate_only`, `blur_noise_light`), which suggests that effective augmentation requires combining multiple augmentation families rather than relying on a single transformation.

e) Comparison with Kaggle competition systems: Table VIII places our best post-augmentation model against publicly documented Kaggle CLDC solutions. The strongest leaderboard systems remain above our result, but the gap is methodologically expected. Their improvements come from system-level competition techniques: ensembling multiple backbones, using 2019/external cassava data, training fold ensembles, applying TTA, relabeling noisy samples, or stacking OOF predictions. In contrast, our final model is a single ViT-B/16 trained under a fixed 3-fold protocol. The most comparable k-fold solutions are TomYanabe, Krut Patel, and kqu7. They exceed our accuracy by about 0.95–1.73 percentage points, but all three use ensembles and/or TTA. This comparison suggests that our post-augmentation single-model result is close to the lower end of competitive Kaggle-era performance, while the remaining performance gap is largely attributable to ensemble-scale inference and additional data rather than to the absence of a fundamentally different backbone.

f) Limitations and future work: Three limitations of the present work are worth stating explicitly. First, we evaluate exclusively under 3-fold stratified cross-validation and do not use a separately held-out test split; out-of-fold validation predictions are reported as a proxy for held-out test performance. While this is statistically meaningful, a fixed external test split would allow a more rigorous comparison with published numbers. Second, the augmentation ablation is conducted on top of a single selected ViT-B/16 configuration; we have not verified whether the same ranking of augmentation policies would hold under other backbones or under significantly different optimizer/scheduler combinations. Third, the input resolution differs across architectures: ResNet-50 and EfficientNet-B4 use 224×224, whereas ViT-B/16 uses 384×384, so part of the ViT advantage may be attributable to higher resolution. Fourth, the experiments are run with single-seed fine-tuning rather than seed averaging, so the reported macro-F1 values include some seed-level variance that we did not characterize. Promising directions for follow-up work include (i) combining the best augmentation policy with light test-time augmentation, (ii) revisiting class-imbalance handling under augmentation, and (iii) adding model-ensemble or distillation steps on top of the Optuna-selected best configurations.

g) Conclusion: Within a unified evaluation protocol and a categorical hyperparameter search, ViT-B/16 fine-tuned from ImageNet weights is the strongest single-model approach on CLDC among the configurations we evaluated, reaching macro-F1 0.7921 ± 0.0068 and accuracy 0.8855 ± 0.0040 when combined with the best augmentation policy (`heavy_combo`). Classical pipelines on raw

pixel statistics are not competitive on this benchmark. Across deep architectures, augmentation and hyperparameter choices contribute as much to the final performance as the choice of backbone, and the gains from augmentation are concentrated where they are most needed—on the minority class. Relative to public Kaggle systems, our result is best interpreted as a controlled single-model baseline: below leaderboard ensembles, but close enough to indicate that much of the gap can plausibly be closed by TTA, fold ensembling, relabeling, and additional data rather than by changing the basic ViT backbone.

References

[1] E. Mwebaze, T. Gebru, A. Frome, S. Nsumba, and J. Tusubira, "iCassava 2019 fine-grained visual categorization challenge," *arXiv preprint arXiv:1908.02900*, 2019.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[3] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[4] J. G. A. Barbedo, "Factors influencing the use of deep learning for plant disease recognition," *Biosystems Engineering*, vol. 172, pp. 84–91, 2018.

[5] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[7] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[8] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[9] ——, "SGDR: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations (ICLR)*, 2017.

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631.

[12] Kaggle and Makerere University AI Lab, "Cassava Leaf Disease Classification," Kaggle Competition, 2020, accessed: 2026-05-18. [Online]. Available: https://www.kaggle.com/competitions/cassava-leaf-disease-classification

[13] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[14] A. Fang, S. Kornblith, and L. Schmidt, "Does progress on ImageNet transfer to real-world datasets?" in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[15] Kaggle and Makerere University AI Lab, "Cassava Leaf Disease Classification: Leaderboard," Kaggle Competition Leaderboard, 2021, accessed: 2026-05-18. [Online]. Available: https://www.kaggle.com/competitions/cassava-leaf-disease-classification/leaderboard

[16] lizh0019, "Cassava: 1st place solution for cassava leaf disease classification," GitHub repository, 2021, accessed: 2026-05-18. [Online]. Available: https://github.com/lizh0019/cassava

[17] D. Stanfield, "cassava-infer," Kaggle notebook, 2021, accessed: 2026-05-18. [Online]. Available: https://www.kaggle.com/code/devonstanfield/cassava-infer

[18] T. Yanabe, "Cassava Leaf Disease Classification: 3rd place solution," GitHub repository, 2021, accessed: 2026-05-18. [Online]. Available: https://github.com/TomYanabe/Cassava-Leaf-Disease-Classification

[19] K. Patel, "My solution for cassava leaf disease classification," Analytics Vidhya / Medium article, 2021, accessed: 2026-05-18. [Online]. Available: https://medium.com/analytics-vidhya/cassava-leaf-disease-classification-kaggle-competition-4407ed1a995d

[20] N. Kozodoi, "Kaggle Leaf Disease Classification: top-1% solution," GitHub repository, 2021, accessed: 2026-05-18. [Online]. Available: https://github.com/kozodoi/Kaggle_Leaf_Disease_Classification

[21] kqu7, "Kaggle-Cassava-Leaf-Disease-Classification," GitHub repository, 2021, accessed: 2026-05-18. [Online]. Available: https://github.com/kqu7/Kaggle-Cassava-Leaf-Disease-Classification

[22] V. Timakin, M. Zhdanov, and E. Tagiev, "Cassava-Leaf-Disease-Classification: Competition pipeline and solution," GitHub repository, 2021, accessed: 2026-05-18. [Online]. Available: https://github.com/vadimtimakin/Cassava-Leaf-Disease-Classification

[23] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.

[24] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

[25] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *International Conference on Learning Representations (ICLR)*, 2018.

[26] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.